



南開大學
Nankai University

密码与网络空间安全学院
物联网通信技术实验报告

具身智能 VLA: $\pi_{0.5}$ 与 StreamingVLA

复现、Profiling 与 Horizon 参数实验

第五组

专业：物联网工程

2026 年 6 月 28 日

目录

摘要	3
1 实验要求、材料来源与完成情况	3
1.1 课程作业要求对照	3
1.2 材料来源与实验边界	4
1.3 报告技术脉络	4
2 VLA 与具身智能背景	5
2.1 从 LLM、VLM 到 VLA	5
2.2 分层决策与端到端 VLA	5
3 π_0: $\pi_{0.5}$ 与 StreamingVLA 的技术基础	6
3.1 模型结构: VLM 骨干与动作专家	6
3.2 Flow Matching 与 action chunk	6
3.3 数据规模与后训练思想	7
4 $\pi_{0.5}$ 论文研读	7
4.1 问题目标: 开放世界家务任务泛化	7
4.2 模型执行流程	7
4.3 训练数据配方	8
4.4 开放世界实验图表分析	8
4.5 $\pi_{0.5}$ 的创新点与局限	10
5 StreamingVLA 论文研读	10
5.1 研究动机: 能力之外的实时性问题	10
5.2 运行时分析与优化目标	10
5.3 动作流匹配: 从动作块到状态增量	11
5.4 自适应提前观测: 动作显著性约束下的观察重叠	12
5.5 LIBERO 主结果与消融数据	13
5.6 运行时分解与视频仿真对比	14
6 源码研读与执行流程核验	16
6.1 openpi 中 $\pi_{0.5}$ -LIBERO 的执行路径	16
6.2 StreamingVLA 对 openpi 的源码扩展	16
6.3 流式推理与 websocket 服务	17
6.4 源码复现适配问题	17
7 复现实验、固定 Horizon Profiling 与自适应探索	18
7.1 课程指定实验协议与本组运行设置	18
7.2 实验过程细化	18

7.3	运行命令与实验步骤	19
7.4	源码增删改说明	20
7.5	profiling 记录实现	21
7.6	短 horizon 队列阻塞定位	22
7.7	h16 边界验证与有效 horizon 范围	22
7.8	固定 horizon 结果汇总	22
7.9	固定 horizon 结果横向与纵向分析	24
7.10	固定 horizon 分任务分析	25
7.11	探索类自适应 horizon 策略设计	25
7.12	自适应 horizon 实现过程	26
7.13	自适应 horizon 实验结果	27
7.14	自适应结果横向与纵向分析	29
8	StreamingVLA 相较 $\pi_{0.5}$ 的改进、优点与局限	29
8.1	执行流程差异	29
8.2	性能收益	30
8.3	技术局限	30
9	总结	30
A	提交材料说明	31

摘要

本实验报告围绕课程大作业指定的具身智能 VLA 主题，完成了 π_0 、 $\pi_{0.5}$ 与 StreamingVLA 的论文研读、源码分析、LIBERO 仿真复现、固定 execution horizon profiling 以及自适应 horizon 探索实验。报告按照“理论理解、源码定位、环境搭建、模型复现、代码修复、实验对比、结果分析”的顺序展开，重点说明本组在复现 StreamingVLA 过程中对环境兼容、源码错误、队列阻塞、horizon 边界和自适应策略实现的排查与修改过程。

复现实验均在 `libero_spatial` 任务集上进行，正式实验规模为 10 个任务、每任务 10 次，共 100 episodes。 $\pi_{0.5}$ 复现成功率为 100%，StreamingVLA baseline 成功率为 96%，平均 episode 时间约为 3.8 s，短于 $\pi_{0.5}$ 的 7.48 s，体现了 StreamingVLA 在流式执行和减少动作等待方面的效率优势。固定 horizon 实验覆盖 baseline、h1、h2、h4、h6、h8、h10。结果显示，h1、h2、h4 作为短 horizon 对照组均未达到成功判定，h6、h8、h10 分别达到 100%、98%、97%，说明当前 StreamingVLA 的执行 horizon 存在明显有效下界，性能在 h4 到 h6 之间出现跃迁。此外，本组对 h16 进行了边界验证。由于当前 checkpoint 的 `action_horizon=10`，单次推理最多返回 10 个动作增量，`replan_steps=16` 会导致 client 在动作队列耗尽后等待，因此 h16 不属于当前模型服务接口可直接支持的同口径固定 horizon 设置。

在探索实验中，本组实现了自适应 horizon 控制器，使系统能够在 h6、h8 和 h10 之间动态选择 execution horizon。由于当前 StreamingVLA server 在真实 rollout 中逐步返回单个动作增量，并不直接暴露完整 action chunk，正式运行采用 `executed_action_variation_proxy` 作为在线一致性代理信号，同时在策略代码中保留 chunk-overlap consistency 的扩展接口。自适应策略完成 100 episodes 真实评估，成功率为 98%，平均 selected horizon 为 7.10，平均推理次数为 15.77；其中 h6、h8、h10 的选择占比分别为 52.1%、39.6%、8.3%。结果表明，该策略并非在所有单项指标上超过固定 horizon，但能够在保持较高成功率的同时减少相对保守设置 h6 的推理次数，体现出动态 horizon 策略在稳定性与效率折中上的探索价值。

关键词：具身智能；视觉语言动作模型； $\pi_{0.5}$ ；StreamingVLA；LIBERO；固定 horizon；自适应 horizon；profiling；实验复现

1 实验要求、材料来源与完成情况

1.1 课程作业要求对照

课程作业并非单纯论文综述，而是围绕 VLA 模型研读、源码分析、平台复现、实验修改和结果分析展开的综合实验。根据课件《VLA 介绍和作业安排》和大作业说明文档，本报告将任务要求整理为表1。其中，固定 horizon profiling 为必做部分，自适应 horizon 为探索部分；两部分均已完成正式实验，并在报告后续章节中给出代码修改、运行结果和图表分析。

表 1: 课程作业要求与本报告完成情况

作业要求	本报告对应内容	完成情况
研读 $\pi_{0.5}$ 与 StreamingVLA 论文、源码	第 4–6 节分析两类模型的结构、执行流程、创新点与局限, 提交包保留关键源码文件	已完成
复现 $\pi_{0.5}$ 与 StreamingVLA	第 7 节记录 LIBERO-Spatial 平台配置、服务端与客户端分离运行方式、100-episode 复现结果、timing 与视频证据	已完成
修改 StreamingVLA 源码并保证可运行	第 7.3–7.4 节说明服务端、客户端、AEO predictor、attention mask 和队列阻塞等源码复现适配过程	已完成
固定 horizon profiling 实验	第 7.5–7.10 节比较 baseline、h1、h2、h4、h6、h8、h10 的成功率、推理次数、完成步数、运行时间和端到端时延	已完成
h16 边界验证	第 7.7 节说明 <code>action_horizon=10</code> 对执行 horizon 的上限约束, h16 作为边界现象分析而非同口径统计组	已完成边界验证
探索自适应 horizon 策略	第 7.11–7.14 节给出基于动作变化代理信号的自适应 horizon 策略, 完成 100 episodes 真实 rollout, 并与固定 h6/h8/h10 对比	已完成
提交实验报告、源码、CSV、图表和日志	提交包包含 <code>source_code</code> 、 <code>data</code> 、 <code>fig</code> 、 <code>README.txt</code> 和实验报告 PDF/LaTeX 源文件	已整理

1.2 材料来源与实验边界

本报告主要依据课程课件、作业说明文档、 $\pi_0/\pi_{0.5}$ 与 StreamingVLA 论文、openpi 与 StreamingVLA 源码, 以及本组在 LIBERO-Spatial 任务集上的实验结果进行整理。论文与源码部分用于分析模型结构、执行流程和系统实现; 实验数据部分来自本组固定 horizon 与自适应 horizon 运行结果, 包括各组 `results.csv`、`timing.txt`、运行日志及由 CSV 重新绘制的结果图。报告中的表格和图像均优先使用可复现实验数据或论文原始结果, 避免只基于主观描述下结论。

h16 作为边界设置进行了验证。由于当前 StreamingVLA checkpoint 的 `action_horizon=10`, server 单次推理最多返回 10 个动作增量, `replan_steps=16` 会使 client 在动作队列耗尽后继续等待。该现象说明 h16 已超出当前模型服务接口支持的 execution horizon 范围, 因此本报告将其作为模型结构边界分析, 而不纳入与 h1–h10 相同口径的 100-episode 固定 horizon 统计。正式固定 horizon 统计采用 $h \in \{1, 2, 4, 6, 8, 10\}$, 其中 h6 是本组额外补充的中间点, 用于观察 h4 到 h8 之间是否存在性能转折。

探索实验部分采用自适应 horizon 策略。由于当前 StreamingVLA server 按流式方式逐步返回动作增量, 并不直接暴露完整 action chunk, 本组在真实 rollout 中使用 `executed_action_variation_proxy` 作为在线一致性代理信号。该策略保留了基于 chunk consistency 的设计思路, 并在当前服务接口条件下实现了可运行的自适应 execution horizon 原型。

1.3 报告技术脉络

本报告围绕“模型原理理解、源码复现修改、固定 horizon 实验、自适应 horizon 探索”四条主线展开。

第一条主线是 VLA 任务闭环。VLA 系统以视觉观测、语言指令和机器人状态作为输入, 经视觉语言模型完成环境语义理解, 再由动作生成模块输出连续控制量, 最终驱动机械臂末端、夹爪等执行器与环境交互。该过程对应课程中强调的“感知–决策–执行”闭环, 也是后续分析 π_0 、 $\pi_{0.5}$ 和 StreamingVLA

的共同基础。

第二条主线是模型能力演进。 π_0 将预训练视觉语言模型与 flow matching 动作专家结合, 使模型能够生成连续高频动作序列; $\pi_{0.5}$ 在此基础上进一步引入异构数据、开放词表视觉语言任务、高层子任务预测和家庭场景后训练, 增强了模型在未见环境中的泛化能力。StreamingVLA 的重点并不在于继续扩大语义泛化范围, 而是面向多阶段 VLA 推理中的执行停顿和端到端延迟问题, 对动作生成、动作发送和重新观测机制进行流式化改造。

第三条主线是源码复现与工程修改。普通 $\pi_{0.5}$ 在一次推理中生成完整 action chunk, client 按块执行; StreamingVLA 则将 action chunk 拆分为逐步去噪、逐步发送和逐步执行的流式过程, 并通过动作状态、剩余动作估计和提前观测逻辑减少等待时间。本组在复现过程中进一步定位并修复了 server 属性访问、StreamingVLA attention mask、AEO predictor 路径、client 队列阻塞等问题, 使模型能够在 LIBERO-Spatial 上稳定运行并达到作业要求的成功率。

第四条主线是 horizon 实验与策略探索。在固定 horizon 实验中, 本组通过修改 replan_steps, 比较 baseline、h1、h2、h4、h6、h8、h10 等设置下的任务成功率、推理次数、完成步数、episode 真实时间和端到端时延, 并分析不同 horizon 对执行稳定性和系统效率的影响。在探索实验中, 本组进一步实现自适应 horizon 策略, 根据在线动作变化信号在 h6、h8 和 h10 之间动态选择执行长度, 用于验证固定 horizon 的局限性以及动态策略的可行性。

2 VLA 与具身智能背景

2.1 从 LLM、VLM 到 VLA

课程课件将具身智能拆解为具身感知、具身决策和具身执行。LLM 主要处理文本符号, VLM 将视觉输入与语言输入对齐, VLA 则在 VLM 的基础上进一步输出机器人可执行动作。VLA 的关键差异在于: 输出不再只是自然语言答案, 而是连续或离散的控制量, 例如末端执行器位姿、关节速度、夹爪开合、底盘速度或动作 token。

具身智能系统并不是孤立地“看图说话”, 而是需要在物理或仿真环境中不断接收新观察、生成动作并修正状态。因此, 同一个高层任务通常会被分解为多个低层步骤, 例如“清理厨房”可能包含“识别碗盘”“打开抽屉”“抓取餐具”“放入水槽”等子任务。VLA 的研究价值正在于把语言目标、视觉上下文和动作控制压缩到统一模型或统一接口中。

2.2 分层决策与端到端 VLA

课件将 VLA 路线概括为工业界分层决策模型与学术界端到端模型。分层模型把感知、规划和技能调用拆开, 通常依赖已有技能库, 优点是执行速度快、工程可控, 缺点是动作空间受限, 复杂泛化依赖技能库覆盖范围。端到端模型用统一模型从观察和语言直接生成动作, 减少模块间误差传递, 但训练数据、算力和推理时延压力更大。

端到端 VLA 又可细分为隐式端到端、显式端到端和分层端到端。隐式端到端模型直接输出动作, 解释性较弱, 但易与大模型骨干结合; 显式端到端模型会生成中间可解释表示, 例如未来图像或轨迹; 分层端到端模型在高层慢思考和低层快执行之间折中。本文重点研读的 $\pi_{0.5}$ 与 StreamingVLA 均属于以隐式动作生成和分层语义推理为核心的路线。

3 π_0 : $\pi_{0.5}$ 与 StreamingVLA 的技术基础

3.1 模型结构：VLM 骨干与动作专家

π_0 的核心贡献是把预训练视觉语言模型骨干与连续动作生成专家结合。图1 显示, π_0 以 PaliGemma/SigLIP 作为视觉语言基础, 接收多视角图像、语言指令和机器人状态, 再通过 action expert 生成未来动作块。动作专家与语言模型骨干并行构成双专家结构, 使模型既保留互联网视觉语言预训练带来的语义理解能力, 又获得机器人动作分布建模能力。

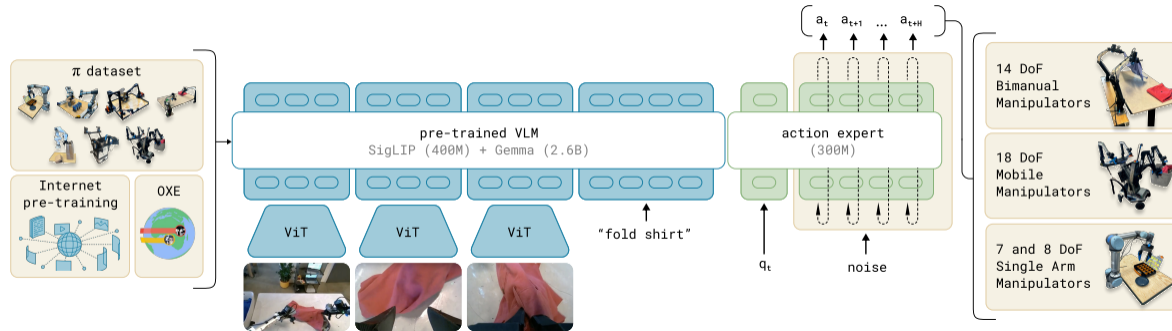


图 1: π_0 框架图: VLM 骨干、机器人状态、动作专家与多机器人数据融合。图源: π_0 论文 Fig.3。

与把动作离散化为词表 token 的 RT-2/OpenVLA 等路线相比, π_0 更强调连续动作分布。对于机械臂控制, 连续动作可以更自然地表达位置、姿态和夹爪开合的细粒度变化; 对于多机器人跨本体训练, 动作维度和状态维度需要被填充、对齐和归一化。

3.2 Flow Matching 与 action chunk

π_0 使用 conditional flow matching 表示动作块分布。训练阶段从真实动作 a 与噪声 ϵ 之间构造中间状态 x_t , 模型学习速度场 v_t , 使其能够从噪声逐步积分到动作。openpi 的 PyTorch 实现中, 普通 $\pi_0/\pi_{0.5}$ 的训练 forward 直接在整块动作上计算:

Listing 1: openpi 中普通 $\pi_0/\pi_{0.5}$ flow matching 损失核心代码, 来源: source_code/openpi_pi05/pi0_pytorch.py

```
1 time_expanded = time[:, None, None]
2 x_t = time_expanded * noise + (1 - time_expanded) * actions
3 u_t = noise - actions
4 ...
5 v_t = self._apply_checkpoint(action_out_proj_func, suffix_out)
6 return F.mse_loss(u_t, v_t, reduction="none")
```

推理阶段则从噪声动作块开始, 以固定步数进行 Euler 积分, 最后一次性返回 H 步动作:

Listing 2: 普通 $\pi_{0.5}$ 推理以动作块为单位返回, 来源: source_code/openpi_pi05/pi0_pytorch.py

```
1 actions_shape = (bsize, self.config.action_horizon, self.config.action_dim)
2 noise = self.sample_noise(actions_shape, device)
3 ...
4 x_t = noise
5 time = torch.tensor(1.0, dtype=torch.float32, device=device)
6 while time >= -dt / 2:
7     v_t = self.denoise_step(state, prefix_pad_masks, past_key_values, x_t, expanded_time)
8     x_t = x_t + dt * v_t
```

```

9   time += dt
10  return x_t

```

这种 action chunk 机制提高了动作生成的稳定性，但也为后续 StreamingVLA 指出的停顿问题埋下原因：系统通常要等完整动作块生成之后才能进入执行，观察、生成和执行之间存在串行等待。

3.3 数据规模与后训练思想

π_0 论文强调大规模机器人数据预训练和高质量后训练的分工。预训练阶段使用多机器人、多任务数据学习通用能力，后训练阶段用目标任务高质量数据提升特定任务表现。图2 展示了 π_0 的数据混合与任务结果图例。该思路对 $\pi_{0.5}$ 很重要： $\pi_{0.5}$ 不只是修改网络结构，而是把数据来源扩展到移动操作、非移动机器人、跨本体实验室数据、高层语义任务、网页多模态数据和人类语言指令。

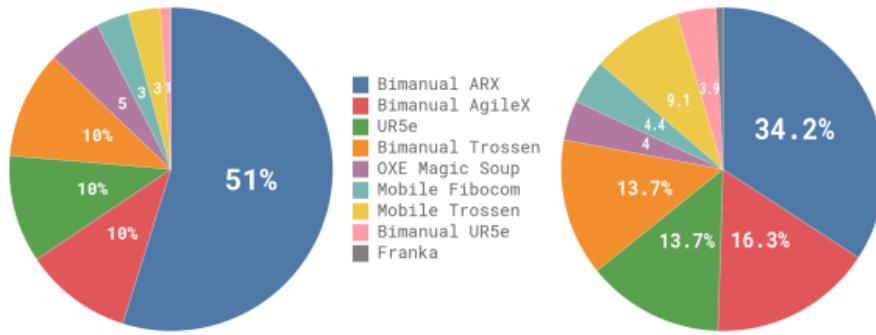


图 2: π_0 数据与训练思想图例。图源： π_0 论文 Fig.4。

4 $\pi_{0.5}$ 论文研读

4.1 问题目标：开放世界家务任务泛化

$\pi_{0.5}$ 的研究目标是让机器人在未见过的真实家庭环境中执行长期、多阶段、语言条件家务任务。与桌面抓取或短时 manipulation 不同，家庭场景具有以下困难：物体类别和背景变化大，任务需要持续数分钟甚至十余分钟，机器人需要在语言目标和当前观察之间不断选择下一子任务，失败可能来自语义理解、定位、抓取、导航或长程顺序规划任一环节。

论文将 $\pi_{0.5}$ 定位为建立在 π_0 基础上的开放世界 VLA。其核心不在于把模型变得更大，而在于重新组织训练数据和推理流程，使同一模型既能预测高层语义子任务，又能生成低层连续控制动作。

4.2 模型执行流程

图3 是 $\pi_{0.5}$ 的核心模型图。它将训练与推理划分为两个阶段：预训练阶段以离散 token 形式统一多种数据源，包括机器人动作、语言子任务、开放词表视觉语言数据等；后训练阶段则面向移动操作任务，在低层动作端加入 flow matching action expert，并将人类语言指导数据纳入训练。

从执行流程看， $\pi_{0.5}$ 的输入包括图像、机器人状态和高层任务指令。模型先根据当前观察和任务目标预测高层子任务，例如从“清理卧室”推断到“拿起枕头”；随后低层动作专家以该子任务作为上下文，生成连续动作块。这个流程对应论文中的高层推理 $\pi_\theta(\hat{\ell} | o_t, \ell)$ 和低层动作推理 $\pi_\theta(a_{t:t+H} | o_t, \hat{\ell})$ 。

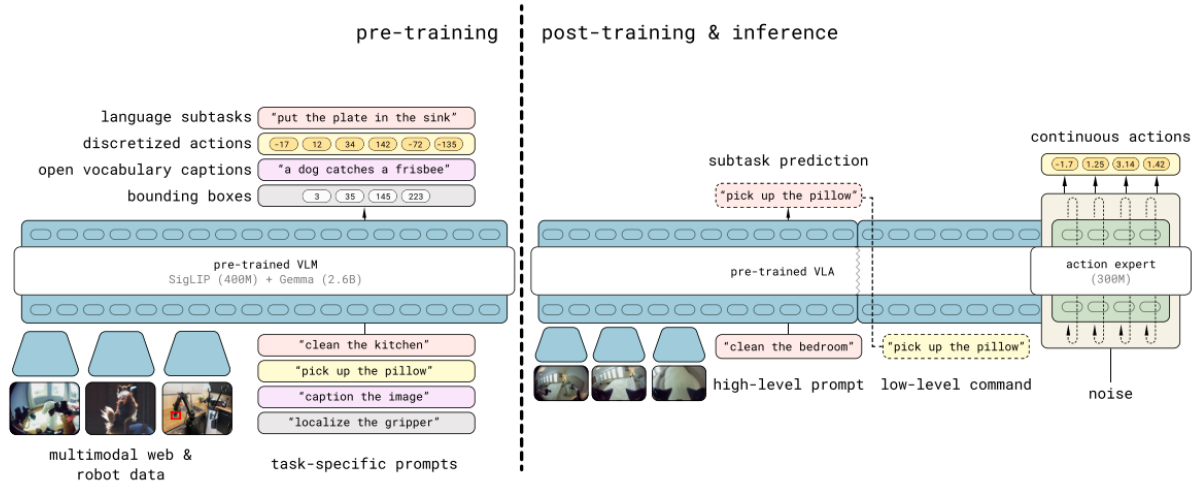


图 3: $\pi_{0.5}$ 模型概览：预训练阶段统一多源离散 token，后训练阶段加入高层子任务推理和连续动作专家。图源： $\pi_{0.5}$ 论文 Fig.3。

4.3 训练数据配方

$\pi_{0.5}$ 的数据配方可概括为六类：移动操作数据 MM、非移动多环境机器人数据 ME、实验室跨本体数据 CE、高层子任务预测 HL、网页多模态数据 WD、后训练阶段的人类语言指导 VI。其设计逻辑如下。

第一，移动操作数据提供目标机器人在家庭环境中的直接经验，但采集成本高，不足以覆盖所有房屋、背景和物体。第二，非移动多环境数据和跨本体数据补充不同场景、不同机器人和不同任务的经验，缓解单一平台数据不足。第三，高层子任务预测把长程任务拆成可执行语义步骤，使模型具备“先决定下一步做什么”的能力。第四，网页多模态数据提高开放词表物体和语言概念理解，尤其对未见物体类别和出分布语言命令有价值。第五，人类语言指导数据让后训练阶段更贴近实际交互过程。

4.4 开放世界实验图表分析

论文在真实家庭与 mock homes 中评估 $\pi_{0.5}$ 。图4 展示真实厨房和卧室中的评估：任务包括把物品放入抽屉、把衣物放入洗衣篮、把餐具放入水槽等。论文图中不仅给出成功进度柱状图，还展示了高层子任务预测随执行步骤变化的轨迹。这说明 $\pi_{0.5}$ 不是一次性输出完整自然语言计划，而是在执行过程中不断用当前观察更新下一子任务。

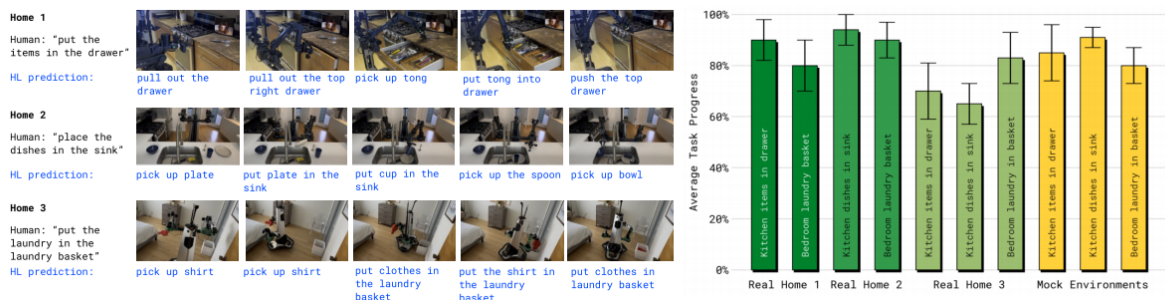


图 4: $\pi_{0.5}$ 真实家庭评估与任务进度。图源： $\pi_{0.5}$ 论文 Fig.7。

图5 进一步说明训练地点数量对泛化的影响。随着训练地点从少量增加到 104 个，mock homes 中四类任务的平均进度提升；语言跟随任务中，无论是 in-distribution 还是 out-of-distribution 物体，follow rate 与 success rate 都随地点数量增加而上升。该实验支撑了论文的一个关键判断：开放世界泛化并非只靠模型规模，还依赖多地点、多物体、多语言指令的覆盖。

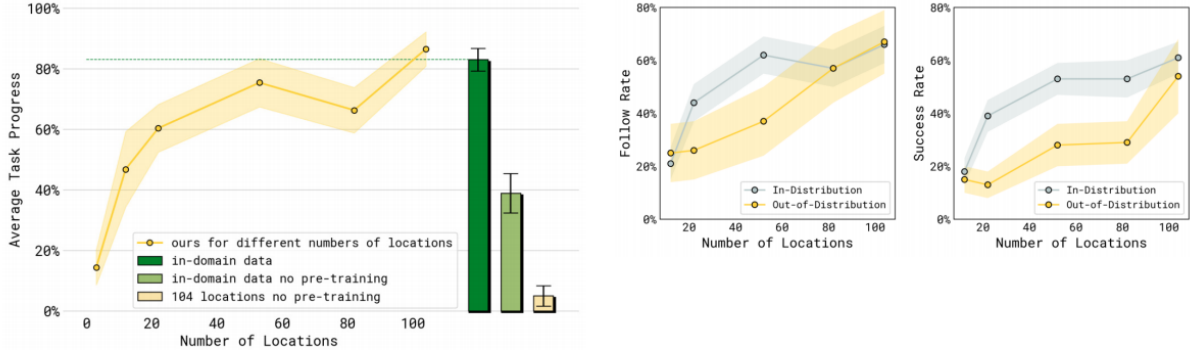


图 5: $\pi_{0.5}$ 训练地点数量与语言跟随能力实验。图源: $\pi_{0.5}$ 论文 Fig.8、Fig.9。

训练配方消融如图6 所示。移除 ME 或 CE 会明显降低 mock homes 任务进度，说明跨环境和跨本体数据对泛化有直接贡献；在语言跟随实验中，移除 WD 对出分布物体影响更明显，说明网页视觉语言数据对开放词表语义理解具有补充作用。与 π_0 和 π_0 -FAST+Flow 相比，完整 $\pi_{0.5}$ 在多项家庭任务上表现更好，说明高层语义数据、网页数据和混合训练流程共同构成改进来源。

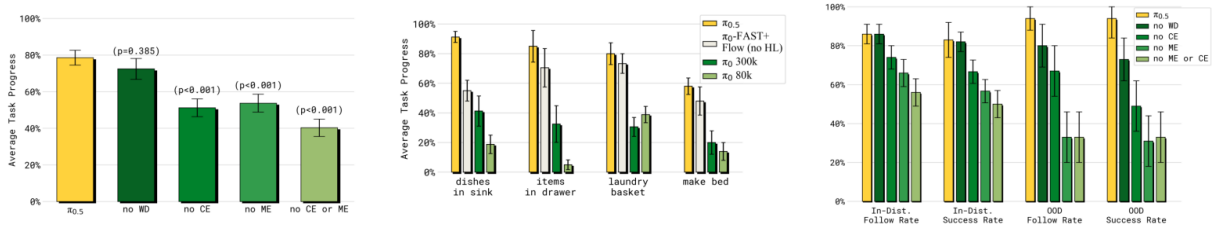


图 6: $\pi_{0.5}$ 训练配方消融及与 π_0 、 π_0 -FAST+Flow 的比较。图源: $\pi_{0.5}$ 论文 Fig.10、Fig.11、Fig.12。

图7 则评估高层推理过程。完整 $\pi_{0.5}$ 在高层和低层都使用同一模型，表现优于无网页数据、无语言指导、隐式高层、无高层数据和 GPT-4 高层策略等对照。该实验说明，高层策略需要结合机器人观察和可执行性经验，单纯调用通用语言模型并不能替代机器人数据训练出的高层推理。

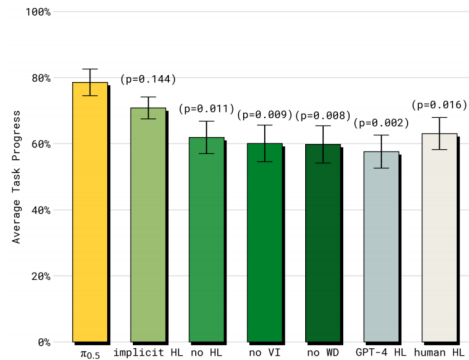


图 7: $\pi_{0.5}$ 高层推理机制评估。图源: $\pi_{0.5}$ 论文 Fig.13。

4.5 $\pi_{0.5}$ 的创新点与局限

$\pi_{0.5}$ 的创新点主要体现在四方面。第一，它把开放世界泛化问题从单纯动作学习扩展到数据配方问题，将机器人数据、高层语义任务和网页多模态数据放入同一训练体系。第二，它采用高层子任务预测加低层连续动作生成的分层执行方式，使长程任务更易被模型分解。第三，它沿用 π_0 的 flow matching action expert，从而保留连续控制能力。第四，它用真实家庭和 mock homes 双重评估，证明模型可以迁移到未见家庭环境。

局限同样明确。首先， $\pi_{0.5}$ 的动作块生成仍存在延迟和停顿，尤其在多阶段模型中，观察、生成和执行之间存在串行等待。其次，开放世界泛化依赖大规模异构数据，数据采集、清洗和后训练成本较高。再次，高层子任务虽提升可解释性和长程控制，但错误高层预测会传递到低层动作。最后，论文中的真实家庭实验任务仍处于特定家务范围，距离完全通用家庭机器人仍有距离。

5 StreamingVLA 论文研读

5.1 研究动机：能力之外的实时性问题

StreamingVLA 不是重新提出一个更大 VLA，而是针对 $\pi_{0.5}$ 等多阶段 VLA 的实时执行问题进行改造。论文指出，传统 action chunking 流程中，模型往往需要先完成观察编码和动作块生成，再执行动作；当下一轮动作生成未完成时，机器人会出现停顿。对于真实机器人，这种停顿不仅降低效率，还会破坏控制流畅性，增加动态环境中的响应风险。

图8直观展示了这一问题：原始 action chunking 存在高延迟和 halting execution；StreamingVLA 则通过 Action Flow Matching 与 Adaptive Early Observation 两个机制，把动作生成、动作执行和下一次观察尽可能重叠，达到低延迟与流畅执行。

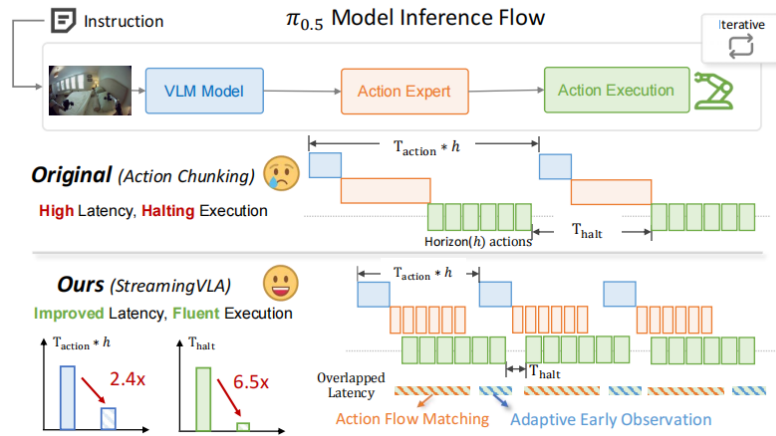


图 8: StreamingVLA 对串行 $\pi_{0.5}$ 推理流程的流式化改造。图源：StreamingVLA 论文 Fig.1。

5.2 运行时分析与优化目标

StreamingVLA 将 VLA 推理拆为三段：观察阶段 T_o 、动作生成阶段 T_g 、动作执行阶段 T_e 。在串行流程中，单动作平均时间和停顿间隔由这些阶段直接叠加决定。论文用 T_{action} 衡量速度，用 T_{halt} 衡量连续执行间的停顿。优化目标不是简单减少单个模型层的计算量，而是增加阶段重叠量 O_{ge} 与 O_{oe} ，分别表示动作生成-执行重叠、观察-执行重叠。其思想可概括为：

$$T_{\text{action}} = \frac{T_o + T_g + T_e - O_{ge} - O_{oe}}{h}, \quad (1)$$

$$T_{\text{halt}} = T_o + T_g - O_{ge} - O_{oe}. \quad (2)$$

图9 给出了完整方法框架：先分析串行流程的时间构成，再分别对动作生成与执行、观察与执行两个方向引入重叠机制。

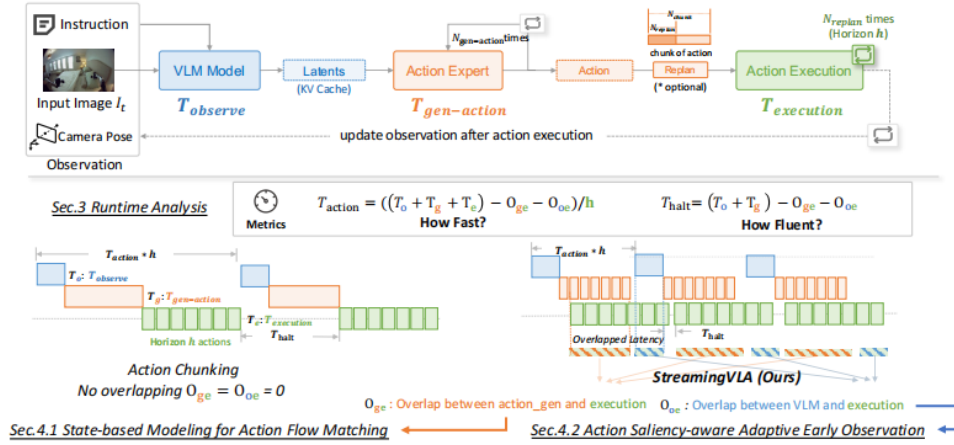


图 9: StreamingVLA 总体方法框架。图源：StreamingVLA 论文 Fig.2。

5.3 动作流匹配：从动作块到状态增量

普通 $\pi_{0.5}$ 的 action expert 在推理时生成整块未来动作，这意味着只有当完整 chunk 得到之后，机器人才能稳定执行。StreamingVLA 的 Action Flow Matching 将建模目标改为状态式动作流：不再只预测绝对动作块，而是维护一个由历史动作累计得到的动作空间状态，模型每一步预测状态更新量。这样，某一步动作一旦生成即可发给执行端，同时下一步动作继续生成。

图10 显示了这种改造。原始动作建模从噪声分布到动作块分布，StreamingVLA 则把动作理解为从初始状态到终态的连续状态更新，强调 prior actions 累计得到的 feature-space state。该机制是实现 generation-execution overlap 的核心。

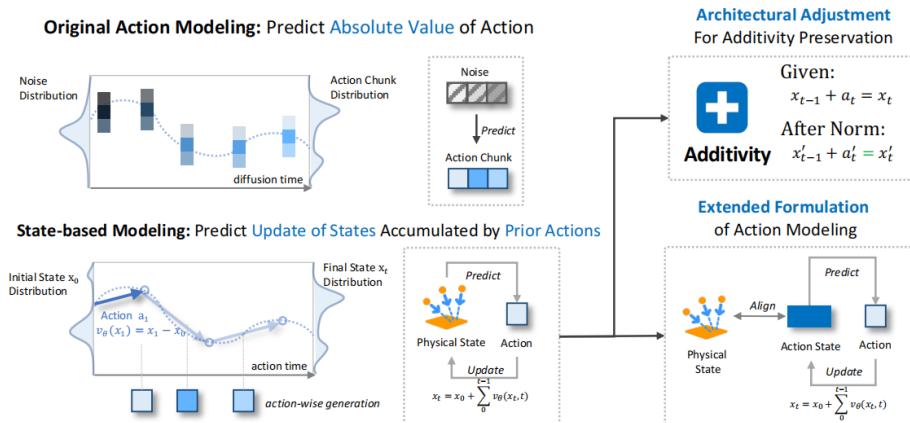


图 10: 状态式动作流匹配示意图。图源：StreamingVLA 论文 Fig.3。

源码中的训练 forward 也体现了这一点：若数据中存在 `action_states`，模型先把初始动作状态与真实动作拼接，再用 `torch.cumsum` 得到累计状态；随后根据时间索引选择相邻状态插值，并把目标速度定义为相邻状态差乘以 `horizon`。

Listing 3: StreamingVLA 状态式 AFM 训练核心，来源：source_code/streamingvla/svla_pytorch.py

```

1 scaled_t = time * self.config.action_horizon
2 index = torch.clamp(scaled_t.floor().long(), 0, self.config.action_horizon - 1)
3 alpha = scaled_t - index.float()
4 if action_states is not None:
5     init_action_state = action_states.unsqueeze(1).to(actions.dtype).to(actions.device)
6     action_states = torch.cat([init_action_state, actions], dim=1)
7     action_states = torch.cumsum(action_states, dim=1)
8 x_t = self.select_by_index(action_states, index) + alpha[:, None, None] * (
9     self.select_by_index(action_states, index+1) - self.select_by_index(action_states, index)
10 )
11 u_t = (self.select_by_index(action_states, index+1) - self.select_by_index(action_states, index)) * self.config.
    action_horizon

```

5.4 自适应提前观测：动作显著性约束下的观察重叠

动作生成与执行重叠后，观察阶段会成为新的瓶颈。如果简单地提前观察，可能在重要动作尚未完成时读取环境，导致观察与真实状态不一致。StreamingVLA 因此提出 Adaptive Early Observation：用轻量 predictor 估计剩余动作对未来视觉 embedding 的影响。如果预测变化较小，允许提前开始下一次观察；如果预测变化较大，则等待动作执行完成，以避免错误观察污染下一轮动作生成。

图11 展示了动作显著性差异：例如拉开抽屉会显著改变环境，此时提前观察风险高；而某些小幅移动对视觉状态影响较小，可以提前观察以减少等待。

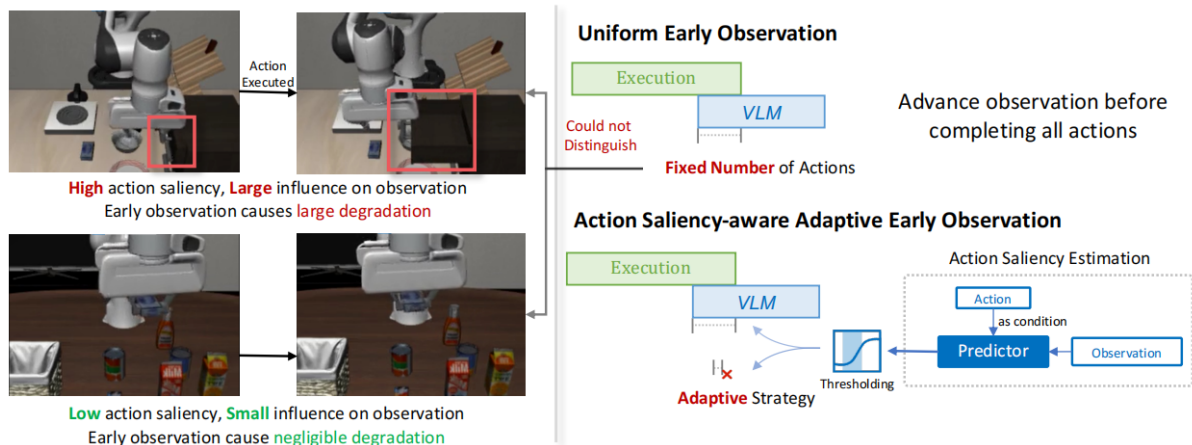


图 11: 动作显著性感知提前观测机制。图源：StreamingVLA 论文 Fig.4。

源码中 AEO predictor 被实现为条件 Transformer。其输入为当前图像 embedding 和剩余动作，输出 $\Delta embedding$ 。推理时计算 $\Delta embedding$ 的二范数，并与阈值比较。

Listing 4: AEO predictor 初始化与 checkpoint 加载，来源：source_code/streamingvla/svla_pytorch.py

```

1 aeo = DiTPredictor(
2     emb_c=2048,
3     emb_len=256,

```



```

4     action_dim=6,
5     n_layers=15,
6     n_heads=16,
7     dim_feedforward=4096,
8     cond_dim=256
9 )
10 checkpoint_path = "TODO: Replace with your_AEO_predictor_checkpoint_path_here"
11 checkpoint = torch.load(checkpoint_path, map_location='cpu')
12 self.aeo_predictor.load_state_dict(real_state_dict)

```

Listing 5: AEO 根据 embedding norm 决定是否提前观测, 来源: source_code/streamingvla/svla_pytorch.py

```

1 delta_embedding = self.aeo_predictor(img_emb, action_tensor)
2 img_emb = img_emb + delta_embedding
3 delta_embedding_norm = torch.linalg.vector_norm(delta_embedding, ord=2).item()
4 ...
5 if delta_embedding_norm > threshold and action_left_sum is not None:
6     yield {"norm_exceeded": True}
7     return

```

5.5 LIBERO 主结果与消融数据

StreamingVLA 的主结果如表2所示。在平均成功率上, $\pi_{0.5}(h=10)$ 为 95.1%, StreamingVLA (AFM+AEO) 为 94.9%, 性能下降小于 0.5 个百分点; 在单动作时间上, 从 49.9 ms 降至 31.625 ms; 在停顿间隔上, 从 230.8 ms 降至 36.0 ms。若以 $\pi_{0.5}(h=5)$ 为基线, AFM+AEO 的时间加速为 2.36 倍, 停顿降低为 6.45 倍。

Method	Success Rate (%) $\uparrow$					Time per Action (ms) $\downarrow$ Halting Gap (ms) $\downarrow$	
	Spatial	Object	Goal	Long	Average	T_{action}	T_{halt}
$\pi_{0.5}(h=5)$	98.8	98.2	98.0	92.4	96.9	74.5	232.3
$\pi_{0.5}(h=10)$	97.4	98.2	96.2	88.6	95.1	49.9 (1.49 $\times$)	230.8 (1.01 $\times$)
RTC (d=1) 5	96.2	19.8	93.0	25.2	58.55	50.2 (1.48 $\times$)	203.6 (1.14 $\times$)
SmolVLA 33	97.0	99.0	97.0	90.0	95.8	51.1 (1.45 $\times$)	180.7 (1.28 $\times$)
VLASH 35	97.5	99.2	97.3	94.6	97.1	40.6 (1.83 $\times$)	-
Temporal Ensembling	97.6	92.4	91.4	78.6	90.0	279.0 (0.26 $\times$)	231.6 (1.003 $\times$)
StreamingVLA (AFM)	97.6	99.0	98.8	93.0	97.1	33.7 (2.21 $\times$)	76.1 (3.05 $\times$)
StreamingVLA (AFM+NEO)	80.6	93.8	92.2	78.2	86.2	29.3 (2.54$\times$)	23.0 (10.10$\times$)
StreamingVLA (AFM+ANAO)	86.0	96.2	93.0	84.8	90.0	30.775 (2.42 $\times$)	27.75 (8.37 $\times$)
StreamingVLA (AFM+AEO)	96.6	96.6	95.4	91.0	94.9	31.625 (2.36 $\times$)	36.0 (6.45 $\times$)

图 12: 论文原始表 1 裁剪图。图源: StreamingVLA 论文 Table 1。

表 2: StreamingVLA 在 LIBERO 上的主结果, 数据源: StreamingVLA 论文 Table 1

方法	Spatial	Object	Goal	Long	Avg.	T_{action} ms	T_{halt} ms
$\pi_{0.5}(h=5)$	98.8	98.2	98.0	92.4	96.9	74.5	232.3
$\pi_{0.5}(h=10)$	97.4	98.2	96.2	88.6	95.1	49.9	230.8
RTC($d=1$)	96.2	19.8	93.0	25.2	58.55	50.2	203.6
SmolVLA	97.0	99.0	97.0	90.0	95.8	51.1	180.7
VLASH	97.5	99.2	97.3	94.6	97.1	40.6	—
Temporal Ensembling	97.6	92.4	91.4	78.6	90.0	279.0	231.6
StreamingVLA(AFM)	97.6	99.0	98.8	93.0	97.1	33.7	76.1
StreamingVLA(AFM+NEO)	80.6	93.8	92.2	78.2	86.2	29.3	23.0
StreamingVLA(AFM+ANAO)	86.0	96.2	93.0	84.8	90.0	30.775	27.75
StreamingVLA(AFM+AEO)	96.6	96.6	95.4	91.0	94.9	31.625	36.0

消融实验如表3 所示。仅有原始 $\pi_{0.5}$ 时成功率为 95.1%，单动作时间 49.9 ms，停顿 230.78 ms；完整 AFM+NM+SA 后成功率达到 97.1%，单动作时间 33.7 ms，停顿 76.1 ms；再加入 Naive EO 虽把时间和停顿降到 29.3 ms 与 23.0 ms，但成功率跌至 86.2%；Adaptive EO 则在 94.9% 成功率下保持 31.625 ms 与 36.0 ms。该结果证明 StreamingVLA 的关键不只是“提前观察”，而是“带动作显著性约束的提前观察”。

表 3: StreamingVLA 技术消融, 数据源: StreamingVLA 论文 Table 2

AFM	NM	SA	Horizon	EO	成功率 (%)	$T_{\text{action}}/T_{\text{halt}}$ ms
—	—	—	10	—	95.1	49.9 / 230.78
✓	—	—	10	—	—	—
✓	✓	—	10	—	61.8	—
✓	✓	✓	10	—	97.1	33.7 / 76.1
✓	✓	✓	10	Naive(2)	86.2	29.3 / 23.0
✓	✓	✓	10	Random(1.4)	90.875	32.125 / 38.3
✓	✓	✓	10	Adaptive(1.4)	94.9	31.625 / 36.0

AFM	NM	SA	Horizon	EO	Success Rate(%)	T_{action} (ms)	T_{halt} (ms)
-	-	-	10	-	95.1	49.9	230.78
✓	-	-	10	-	-	-	-
✓	✓	-	10	-	61.8	-	-
✓	✓	✓	10	-	97.1	33.7	76.1
✓	✓	✓	10	Naive (2)	86.2	29.3	23.0
✓	✓	✓	10	Random (1.4)	90.875	32.125	38.3
✓	✓	✓	10	Adaptive (1.4)	94.9	31.625	36.0

图 13: 论文原始表 2 裁剪图。图源: StreamingVLA 论文 Table 2。

5.6 运行时分解与视频仿真对比

论文附录的运行时分分解进一步说明加速来源。图14 中, $\pi_{0.5}$ 的观察、生成和执行几乎串行排列; StreamingVLA 通过重叠压缩等待区间。论文附录文本给出一组实际 profiling 数值: 基线观察时间

$T_o = 58$ ms、生成时间 $T_g = 180$ ms、执行单步时间 $T_e/h = 27$ ms；StreamingVLA 通过 O_{ge} 与 O_{oe} 重叠，报告 2.40 倍 latency 改善和 6.71 倍 halting gap 改善。

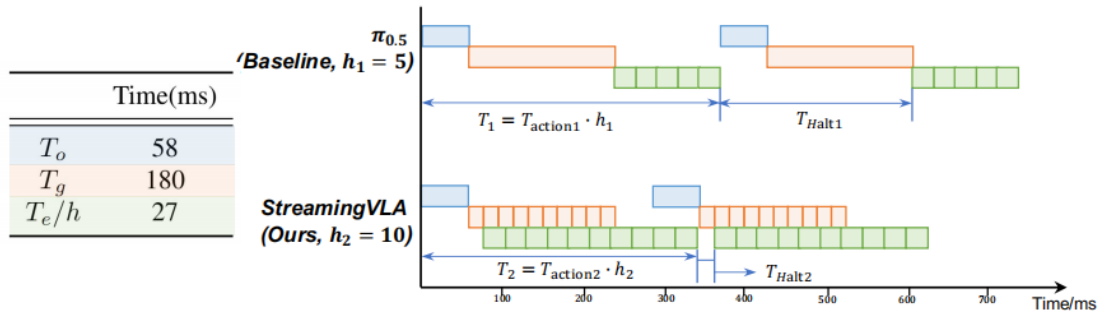


图 14: StreamingVLA 与 $\pi_{0.5}$ 运行时分解。图源：StreamingVLA 论文 Fig.6。

图15 是论文给出的同任务视频仿真截图对比。Goal 任务中，StreamingVLA 在 16 秒内完成开抽屉并放碗， $\pi_{0.5}$ 在 20 秒时尚未拿起碗；Long 任务中，StreamingVLA 在 20 秒时已经完成两个物体的 pick-and-place， $\pi_{0.5}$ 仍在接近第二个物体。该图可作为课程要求“同一任务下生成执行视频对比”的论文证据。本组在提交包中进一步补充了 libero_spatial 任务上的实际运行视频截图和结果记录，用于与论文中的同任务视频证据形成对应。

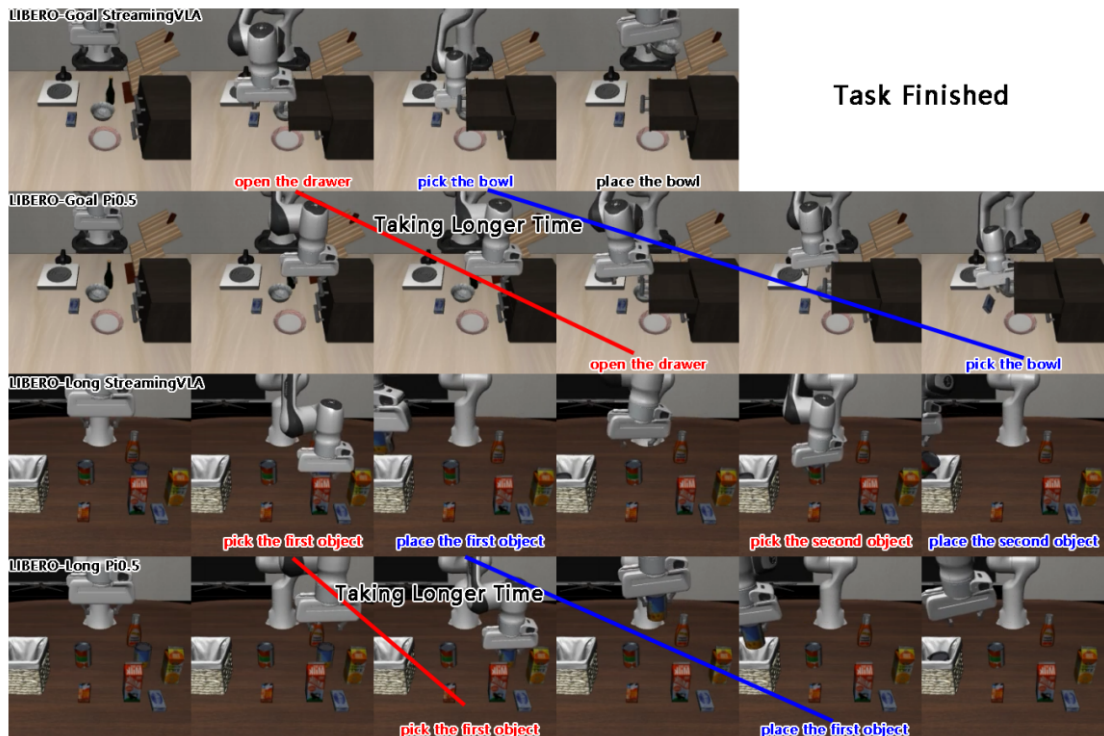


图 15: 论文附录中的同任务视频仿真截图比较。图源：StreamingVLA 论文 Fig.8。

6 源码研读与执行流程核验

6.1 openpi 中 $\pi_{0.5}$ -LIBERO 的执行路径

openpi README 显示, $\pi_{0.5}$ -LIBERO 是官方提供的 fine-tuned checkpoint, 配置名为 pi05_libero。训练配置中, 该模型使用 PiOConfig(pi05=True, action_horizon=10, discrete_state_input=False), 数据集为 physical-intelligence/libero, 权重从 pi05_base 加载。

Listing 6: openpi $\pi_{0.5}$ -LIBERO 配置摘录, 来源: source_code/openpi_pi05/config.py

```

1 TrainConfig(
2     name="pi05_libero",
3     model=pi0_config.PiOConfig(pi05=True, action_horizon=10, discrete_state_input=False),
4     data=LeRobotLiberoDataConfig(
5         repo_id="physical-intelligence/libero",
6         base_config=DataConfig(prompt_from_task=True),
7         extra_delta_transform=False,
8     ),
9     batch_size=16,
10    weight_loader=weight_loaders.CheckpointWeightLoader(
11        "gs://openpi-assets/checkpoints/pi05_base/params"
12    ),
13 )

```

LIBERO 输入变换由 LiberoInputs 完成。普通 openpi 版本主要将 observation/image、wrist_image、observation/state 和 prompt 映射到模型输入; 输出变换 LiberoOutputs 只取动作前 7 维, 因为 LIBERO 环境使用 7 维动作。

6.2 StreamingVLA 对 openpi 的源码扩展

StreamingVLA 在 openpi 基础上添加了 SVLA 模型类型、SVLA 配置标志、动作状态输入、剩余动作输入、阈值输入和流式 websocket server。其配置名为 streamingvla_pi05_libero, 关键差异如代码所示:

Listing 7: StreamingVLA 训练配置摘录, 来源: source_code/streamingvla/config.py

```

1 TrainConfig(
2     name="streamingvla_pi05_libero",
3     model=pi0_config.PiOConfig(pi05=True, svla=True, action_horizon=10, discrete_state_input=False),
4     data=LeRobotLiberoDataConfig(
5         repo_id="/path/to/libero_dataset_with_action_states",
6         base_config=DataConfig(prompt_from_task=True),
7         extra_delta_transform=False,
8         use_action_states=True,
9     ),
10    batch_size=16,
11    weight_loader=weight_loaders.CheckpointWeightLoader(
12        "gs://openpi-assets/checkpoints/pi05_libero/params"
13    ),
14    pytorch_weight_path="/path/to/pi05_libero_pytorch_checkpoint",
15    num_train_steps=100_000,
16 )

```

模型基础类中, StreamingVLA 版本的 Observation 增加了 action_states、action_left_sum 和 threshold 三个字段; PiOConfig 增加 svla 布尔值, 当 pi05=True 且 svla=True 时返回 ModelType.SVLA。这说明 StreamingVLA 的改造并非只在推理脚本层面, 而是贯穿配置、数据、模型和服务端。

6.3 流式推理与 websocket 服务

普通 $\pi_{0.5}$ 的 `infer` 一次返回动作块；StreamingVLA 增加 `streaming_infer`，将模型的 generator 输出逐个包装后返回。模型端的 `sample_actions` 每次生成状态差分 `result = x_t - temp_x_t` 并 `yield result`，服务端再逐条发送给客户端。

Listing 8: StreamingVLA 逐动作输出，来源：source_code/streamingvla/svla_pytorch.py

```
1 while action_index < self.config.action_horizon:
2     temp_x_t = x_t.detach().clone()
3     v_t = self.denoise_step(state, prefix_pad_masks, past_key_values, x_t, expanded_time)
4     expanded_time += dt
5     x_t = x_t + dt * v_t
6     action_index += 1
7     result = x_t - temp_x_t
8     yield result
```

Listing 9: 策略封装层的流式输出，来源：source_code/streamingvla/policy.py

```
1 observation = _model.Observation.from_dict(inputs)
2 action_generator = self._sample_actions(sample_rng_or_pytorch_device, observation, **sample_kwargs)
3 for action_token in action_generator:
4     if isinstance(action_token, dict) and action_token.get("norm_exceeded"):
5         yield action_token
6     else:
7         outputs = {"state": inputs["state"], "actions": action_token, "action_states": torch.zeros_like(action_token)}
8         outputs = self._output_transform(outputs)
9         yield outputs
```

服务端需要根据 `policy` 内部模型的配置判断是否启用 `streaming server`。原始的脚本会直接访问 `policy.model.config.svla`，但实际 `Policy` 对象中模型保存在私有成员 `_model`，因此会触发属性访问错误。本组修复后先取得真实 `policy` 对象，再访问其模型配置：

Listing 10: 修复后的 `streaming server` 选择逻辑，来源：experiment_modifications/serve_policy.py

```
1 actual_policy = policy._policy if isinstance(policy, _policy.PolicyRecorder) else policy
2 use_streaming_server = actual_policy._model.config.svla
3
4 if not use_streaming_server:
5     server = websocket_policy_server.WebsocketPolicyServer(...)
6 else:
7     server = streaming_websocket_policy_server.WebsocketPolicyServer(...)
8 server.serve_forever()
```

6.4 源码复现适配问题

StreamingVLA 源码并非开箱即用，复现前需要结合本地环境、checkpoint 路径和 LIBERO 数据路径进行适配。本组重点检查了模型服务端、AEO predictor、数据预处理脚本和动作维度设置，确认需要修改的内容主要集中在三类：一是将源码中的占位路径替换为实际 checkpoint 与数据路径；二是根据源码真实参数定义修正运行命令；三是检查 AEO predictor 与 LIBERO 动作维度之间的一致性。

具体而言，`svla_pytorch.py` 中 AEO predictor 的 `checkpoint_path` 需要替换为实际权重路径；`streamingvla_pi05_libero` 配置中的 `repo_id` 与 `pytorch_weight_path` 需要指向本地预处理后的数据和模型权重；`preprocess_dataset.py` 实际使用位置参数 `dataset_dir`，因此运行时应采用 `python scripts/preprocess_dataset.py /path/to/your/1erobot/data`。此外，predictor 训练脚本

本中的模型导入文件名和 AEO predictor 的 6 维动作输入也需要与实际源码和官方 checkpoint 保持一致。上述检查保证了后续 server 启动、streaming 推理和 LIBERO rollout 能够在同一套环境中稳定执行。

7 复现实验、固定 Horizon Profiling 与自适应探索

7.1 课程指定实验协议与本组运行设置

本组以 `examples/libero/streamingvla.py` 中的 `replan_steps` 作为主要实验变量，在不改变模型主干、checkpoint 权重、任务定义和成功率判定逻辑的前提下，完成固定 horizon profiling 与自适应 horizon 探索实验。

正式实验统一使用 `libero_spatial`，每组包含 10 个任务、每任务 10 次，共 100 episodes。固定 horizon 组包括 baseline、h1、h2、h4、h6、h8、h10，其中 baseline 保留原始 StreamingVLA 的 AEO judge 行为，其余固定组关闭 judge，使 `replan_steps` 更接近真实 execution horizon。自适应实验在固定结果基础上选择 {6, 8, 10} 作为候选 horizon，并完成 100 episodes 真实 rollout，用于验证动态 execution horizon 调度的可行性。

7.2 实验过程细化

为了使复现实验可检查、可复现，本组将实验过程拆分为九个阶段，每一阶段均对应明确的输入、操作和输出，如表4所示。

表 4: 复现实验流程、关键修改与产出物

阶段	目标	具体操作	产出物
需求解析	明确必做与探索指标	根据作业说明确认最低记录字段： <code>model_inference_time_ms</code> 、 <code>total_latency_ms</code> 、 <code>inference_count</code> 、 <code>task_completion_steps</code> 、 <code>episode_wall_time_ms</code> 、 <code>task_success</code> 、 <code>horizon</code>	指标定义与结果表字段
环境分离	保证 server 与 client 均可运行	server 使用项目根目录运行模型服务；client 使用 <code>examples/libero/.venv</code> 运行 LIBERO、MuJoCo 和评估脚本，避免依赖版本冲突	双环境运行流程
模型服务启动	加载 StreamingVLA checkpoint	在 GPU 环境中启动 <code>scripts/serve_policy.py</code> ，指定 <code>streamingvla_pi05_libero</code> 配置和 LIBERO checkpoint	websocket server 指定
基线复现	确认原始系统可用	分别运行 $\pi_{0.5}$ 与 StreamingVLA baseline，保存 timing、日志和视频证据	$\pi_{0.5}$ 100/100, StreamingVLA 96/100

阶段	目标	具体操作	产出物
源码修复	解决官方源码复现问题	修复 <code>policy.model</code> 访问、 <code>use_sfp</code> 不存在、AEO predictor 路径、短 horizon 队列阻塞等问题	可稳定运行的 StreamingVLA
profiling 改造	为每个 episode 写出 CSV	在 <code>streamingvla.py</code> 中加入 <code>profiling_csv_path</code> 、 <code>method_name</code> 、 <code>fixed_horizon_mode</code> ，并在 episode 结束时写出统计行	各组 <code>results.csv</code>
固定扫参	比较不同 execution horizon	运行 baseline、h1、h2、h4、h6、h8、h10；h16 单独定位为超出当前 action chunk 上限	固定 horizon 统计表和 4 张图
自适应探索	根据动作稳定性动态选择 horizon	新增 <code>adaptive_horizon_policy.py</code> 和相关 <code>runner/analysis</code> ，在 h6/h8/h10 间在线选择	100 episodes adaptive 结果
分析归档	横向、纵向分析结果	合并 CSV，按 method 和 task 两个维度统计，绘制固定 horizon 与 adaptive 对比图	更新报告与压缩包

上述过程体现了本组对实验的修改思考：先复现原始系统以得到可比较基线，再只围绕 horizon 执行逻辑和 profiling 记录做最小改动；遇到 h1-h4 的异常表现和 h16 边界验证后，进一步回到 server/client 队列机制与模型 `action_horizon` 进行定位，而不是简单将其归为随机波动。

7.3 运行命令与实验步骤

server 端负责加载 StreamingVLA 模型并通过 websocket 流式返回动作。client 端负责 LIBERO 环境交互、动作执行、视频保存和 profiling 记录。两端分离运行，能够避免 LIBERO Python 3.8 依赖与 server 侧新 PyTorch 环境互相污染。

Listing 11: StreamingVLA server 启动命令

```

1 cd ~/autodl-tmp/StreamingVLA
2 source .venv/bin/activate
3 CUDA_VISIBLE_DEVICES=0 python scripts/serve_policy.py policy:checkpoint \
4   --policy.config=streamingvla_pi05_libero \
5   --policy.dir=~/autodl-tmp/StreamingVLA/checkpoints/StreamingVLA_LIBERO/58300 \
6   2>&1 | tee runs/server.log

```

固定 horizon 组通过显式设置 `replan_steps` 和 `fixed_horizon_mode` 完成。以 h8 为例，命令如下。

Listing 12: 固定 horizon h8 评估命令

```

1 source examples/libero/.venv/bin/activate
2 export PYTHONPATH=$PYTHONPATH:~/autodl-tmp/StreamingVLA/third_party/libero
3 export MUJOCO_GL=egl
4 export PYOPENGL_PLATFORM=egl
5 export MUJOCO_EGL_DEVICE_ID=0
6
7 python examples/libero/streamingvla.py \
8   --args.host 127.0.0.1 --args.port 8192 \

```

```

9  --args.task-suite-name libero_spatial \
10 --args.num-trials-per-task 10 --args.seed 7 \
11 --args.replan-steps 8 --args.fixed-horizon-mode \
12 --args.method-name h8 \
13 --args.profiling-csv-path runs/fixed_horizon/h8/results.csv \
14 --args.video-out-path runs/fixed_horizon/h8/videos \
15 --args.timing-output-path runs/fixed_horizon/h8/timing.txt \
16 2>&1 | tee runs/fixed_horizon/h8/log.txt

```

自适应探索组在命令层面同时开启 `adaptive_horizon_mode` 与 `fixed_horizon_mode`。这里的 `fixed_horizon_mode` 并不表示 horizon 固定，而是用于关闭 AEO judger，避免 judger 在中途提前触发重观测，从而保证 horizon 变化只来自自适应策略本身。

Listing 13: 自适应 horizon 真实 rollout 命令

```

1 python examples/libero/streamingvla.py \
2   --args.host 127.0.0.1 --args.port 8192 \
3   --args.task-suite-name libero_spatial \
4   --args.num-trials-per-task 10 --args.seed 7 \
5   --args.replan-steps 8 \
6   --args.adaptive-horizon-mode \
7   --args.fixed-horizon-mode \
8   --args.adaptive-min-horizon 6 \
9   --args.adaptive-mid-horizon 8 \
10  --args.adaptive-max-horizon 10 \
11  --args.adaptive-tau-low 0.03 \
12  --args.adaptive-tau-high 0.07 \
13  --args.method-name adaptive_chunk_consistency \
14  --args.profiling-csv-path runs/adaptive_horizon/adaptive_chunk_consistency/adaptive_horizon_results.csv

```

7.4 源码增删改说明

本组没有修改模型主干结构、checkpoint 权重、LIBERO 任务定义和成功率判定逻辑。修改集中在四类文件：官方复现修复、profiling 记录、固定 horizon 执行语义、自适应策略。表5 给出逐文件说明。

表 5: 源码增删改说明

文件	修改内容与作用
<code>serve_policy.py</code>	修改 server 类型判断逻辑。原代码直接访问 <code>policy.model.config.svla</code> ，但实际模型保存在 <code>policy</code> 内部的 <code>_model</code> 中；修复后可正确识别 StreamingVLA 模型并启动 streaming websocket server。
<code>svla_pytorch.py</code>	修复 <code>use_sfp</code> 配置缺失、attention mask 构造和 AEO predictor checkpoint 路径问题，使 StreamingVLA 能够完成流式动作生成和 AEO 相关推理。
<code>streaming_websocket_client_policy.py</code>	在清空动作队列后补充 <code>not_full.notify_all()</code> ，解决短 horizon 下接收线程阻塞问题，使 h1/h2/h4 能完整产生 100-episode CSV，保证短 horizon 对照结果有效。

文件	修改内容与作用
streamingvla.py	增加 profiling_csv_path、method_name、fixed_horizon_mode、adaptive_horizon_mode 等参数，并记录 episode 级指标，支撑固定 horizon profiling 与自适应 horizon 真实 rollout。
fixed_horizon_runner.py	新增固定 horizon 批量运行脚本，用于统一运行 baseline、h1、h2、h4、h6、h8、h10，并保存命令、日志和 CSV，保证各组实验流程一致。
latency_analysis.py	新增统计分析脚本，用于合并各组 results.csv，计算成功率、推理次数、完成步数、运行时间和时延，并自动生成固定 horizon 统计表和结果图。
adaptive _horizon_policy.py	新增自适应 horizon 策略，实现 h6/h8/h10 三档选择，并提供 executed-action variation 在线代理信号，在当前 server 接口条件下实现可运行的自适应 horizon controller。
adaptive _horizon_runner.py	新增自适应实验运行脚本，用于统一候选 horizon、阈值、输出路径和日志记录，保证探索实验能够按固定参数复现。
adaptive _horizon_analysis.py	新增自适应实验分析脚本，用于对比 adaptive 与固定 h6/h8/h10，绘制成功率、推理次数、episode 时间和 selected horizon 分布图，支撑探索题结果分析。

7.5 profiling 记录实现

streamingvla.py 的 profiling 采用 episode 级写出方式。每次 episode 内部维护 infer_times_ms、latency_times_ms、obs_times 和动作步数，episode 结束后写入一行 CSV。这样既能覆盖作业最低要求，又不会产生过大的逐步日志文件。

Listing 14: 新增 profiling 字段

```

1 _PROFILING_FIELDS = [
2     "episode_id", "task_id", "task_name", "horizon", "method", "seed",
3     "inference_count", "task_completion_steps", "episode_wall_time_ms",
4     "task_success", "avg_model_inference_time_ms", "avg_total_latency_ms",
5     "selected_horizon_mean", "selected_horizon_min", "selected_horizon_max",
6     "selected_horizon_sequence", "consistency_error_mean", "consistency_error_max",
7     "gripper_change_count", "adaptive_signal_source", "result_source",
8 ]

```

推理耗时和端到端时延的起止点如下。需要说明的是，StreamingVLA 是流式 websocket 架构，client.infer() 更多表示“提交当前 observation 并触发 server 推理”，真正的模型计算与动作接收发生在后台流式流程中。因此，报告同时记录 avg_model_inference_time_ms 与 avg_total_latency_ms，其中后者更接近机器人从 observation 就绪到拿到可执行 action 的实际等待。

Listing 15: 推理耗时与端到端时延记录位置

```

1 t_latency_start = time.monotonic()
2 ...
3 t_infer_start = time.monotonic()
4 client.infer(element, clear_queue)
5 infer_times_ms.append((time.monotonic() - t_infer_start) * 1000)
6 ...

```



```

7 action_data = client.get_next_action(timeout=20)
8 latency_times_ms.append((time.monotonic() - t_latency_start) * 1000)

```

固定 horizon 与自适应 horizon 都要求丢弃上一轮未执行完的过期动作，否则短 horizon 会继续消费旧 chunk 中与新 observation 不一致的动作。对应修改如下。

Listing 16: 固定和自适应模式下清空过期动作队列

```

1 clear_queue = new_task or args.fixed_horizon_mode or args.adaptive_horizon_mode
2 client.infer(element, clear_queue)

```

7.6 短 horizon 队列阻塞定位

固定 horizon 实验最重要的工程问题出现在 h1。复现早期，h1 的第 1 个 episode 可以跑完，但进入第 2 个 episode 后出现 Queue Is Full ! 并长时间无输出。结合 server 日志可知 server 没有 traceback，问题不在模型崩溃，而在 client 侧动作接收队列。

根因是生产速度与消费速度不匹配。server 每次流式推理固定产生 10 个动作增量；h1 每轮只消费 1 个动作后立即重新观测，净剩余 9 个动作进入队列。队列容量为 100，运行十余轮后接收线程阻塞在 put()。episode 边界处，主线程执行 queue.clear() 只清空 deque，却没有唤醒正在等待 not_full 条件变量的生产者，后台接收线程无法恢复，主线程再等待 get_next_action，最终形成阻塞。

Listing 17: 队列阻塞修复：清空后唤醒生产者

```

1 with self._action_queue.mutex:
2     self._action_queue.queue.clear()
3     self._action_queue.not_full.notify_all()

```

该修复是本组实验中的关键修改之一。修复后 h1、h2、h4 均能完整运行 100 episodes 并产生 CSV。虽然这些短 horizon 设置未达到任务成功判定，但它们提供了重要的下界对照：当 execution horizon 过短时，模型频繁重置观测与动作状态，流式去噪尚未积累到稳定动作阶段，任务难以完成。由此可以区分“程序阻塞”和“短 horizon 语义不足”两类问题。

7.7 h16 边界验证与有效 horizon 范围

作业说明建议比较 $h=16$ ，用于观察过长 horizon 是否导致动作过时或任务表现下降。本组将 h16 作为边界设置进行了验证。实验定位发现，当前 checkpoint 的 action_horizon=10，server 单次推理最多只产生 10 个流式动作增量。若 client 设置 replan_steps=16，它会在第 10 个动作之后继续等待第 11–16 个动作，但 server 不会自动补发超出动作块长度的增量。

因此，本报告将 h16 定位为模型结构和服务接口边界，而不是与 h1–h10 相同口径的固定 horizon 统计组。若要真正评估 h16，需要将模型动作块长度扩展到 16 或以上，并配套重新训练或适配权重、server 推理循环和 client 队列消费逻辑。由于本次作业的核心是修改 execution horizon 而非重新训练动作块模型，正式统计采用 $h \in \{1, 2, 4, 6, 8, 10\}$ 。其中 h6 是本组额外补充的中间点，用于观察 h4 到 h8 之间是否存在性能转折。

7.8 固定 horizon 结果汇总

表6 汇总了 baseline 与固定 horizon 结果。baseline 保留原始 AEO judger；h1–h10 均关闭 judger，以保证横向比较中的主要差异来自 execution horizon。

表 6: 固定 execution horizon profiling 结果汇总, 任务集为 `libero_spatial`

方法	horizon	episodes	成功率	平均推理次数	平均步数	平均时间 (s)	平均端到端时延 (ms)
baseline	10	100	96%	16.62	120.92	3.82	4.67
h1	1	100	0%	220.00	220.00	38.18	112.54
h2	2	100	0%	110.00	220.00	18.64	47.69
h4	4	100	0%	55.00	220.00	8.39	10.97
h6	6	100	100%	18.51	108.78	4.32	13.46
h8	8	100	98%	14.14	109.43	3.99	10.21
h10	10	100	97%	11.45	110.39	3.83	8.35

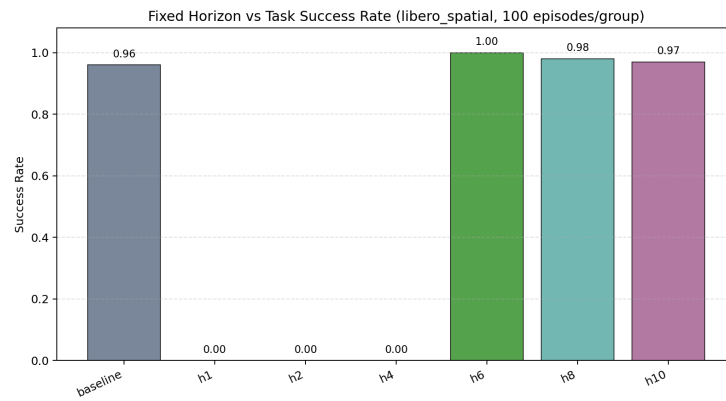


图 16: 不同固定 horizon 下任务成功率对比。h1、h2、h4 均为 0%，h6 后出现明显跃迁。

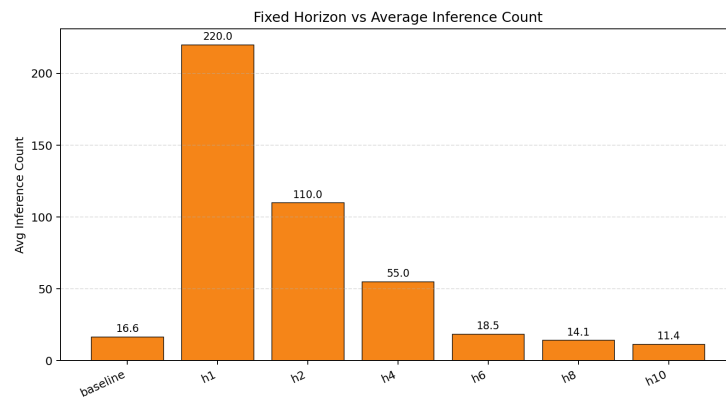


图 17: 不同固定 horizon 下平均模型调用次数对比。horizon 越小, 重新观测和调用模型越频繁。



图 18: 不同固定 horizon 下平均完成步数与真实运行时间对比。h1、h2、h4 均达到 220 步上限。

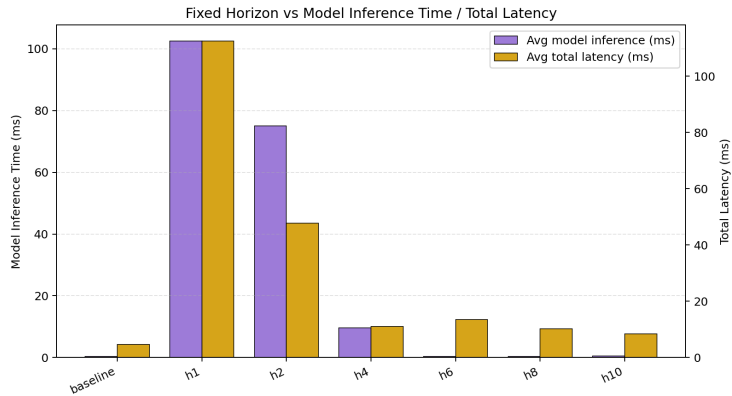


图 19: 不同固定 horizon 下模型提交耗时与端到端时延对比。短 horizon 因频繁重规划产生显著额外负担。

7.9 固定 horizon 结果横向与纵向分析

固定 horizon 实验表明，StreamingVLA 的 execution horizon 并不是单调意义上的“越短越稳”或“越长越快”。在 h1、h2、h4 三个短 horizon 设置下，系统虽然能够完成完整 100-episode 运行并产生有效 profiling 记录，但任务均未达到成功判定，平均步数达到 220 步上限。这说明过短 horizon 会使模型频繁重新观测和重置动作状态，导致 flow matching 的动作增量尚未积累到稳定执行阶段便被中断。

从 h4 到 h6，成功率由 0% 跃升至 100%，平均步数从 220 降至 108.78，说明当前 StreamingVLA checkpoint 的有效 execution horizon 下界位于 4 与 6 之间。该跃迁是本次固定 horizon profiling 的核心发现：较短 horizon 虽然带来更高反馈频率，但如果低于模型流式去噪所需的最小执行长度，反而会破坏动作连续性。

在 h6、h8、h10 区间内，系统进入高成功率平台。h6 成功率最高，体现出更频繁观测带来的稳定性；h10 推理次数最少、平均运行时间最低，体现出较长 horizon 的效率优势；h8 位于两者之间，表现出较好的折中性。因此，固定 horizon 不存在对所有任务和阶段都最优的单点设置，这也为后续自适应 horizon 策略提供了直接实验依据。

从推理次数看，h1 平均 220 次，h2 为 110 次，h4 为 55 次，h8 为 14.14 次，h10 为 11.45 次，基本符合“每次推理执行更多动作，则 episode 内推理调用减少”的预期。h6 为 18.51 次，在推理频率上明显低于 h1/h2/h4，同时保持最高成功率，是本次固定实验中偏可靠性的最优点。h10 的推理次数

最少，平均时间也最低，但成功率低于 h6，说明效率提升和鲁棒性之间存在折中。

从完成步数和运行时间看，h1、h2、h4 均达到 220 步上限，说明这些短 horizon 设置并非“慢速完成”，而是在最大步数限制内仍未达到任务成功判定。h6、h8、h10 的平均步数分别为 108.78、109.43、110.39，差异很小，说明进入稳定区间后，任务完成路径长度基本相近。运行时间则呈现 h10 最短、h8 次之、h6 略长的趋势，主要原因是 h10 的推理调用最少，重观测和等待次数更少。

7.10 固定 horizon 分任务分析

表7 同时列出 h6/h8/h10 与自适应策略在 10 个任务上的成功次数。固定 horizon 中，h6 在所有任务上均为 10/10；h8 主要在“next to the ramekin”任务下降到 8/10；h10 则在“next to the ramekin”与“wooden cabinet”两个空间关系更敏感的任务上分别为 9/10 和 8/10。这说明长 horizon 的主要风险不是全局失效，而是在少数需要更精细空间调整的任务上容易受到观测过时影响。

表 7: 固定 horizon 与 adaptive 的分任务成功次数，每个任务 10 次

ID	任务简写	h6	h8	h10	adaptive	adaptive 平均推理	adaptive 平均 h
0	between plate and ramekin	10	10	10	10	13.20	6.56
1	next to ramekin	10	8	9	9	17.20	7.30
2	from table center	10	10	10	10	14.50	7.02
3	on cookie box	10	10	10	10	13.70	6.63
4	top drawer of cabinet	10	10	10	10	17.90	7.38
5	on ramekin	10	10	10	9	17.00	6.76
6	next to cookie box	10	10	10	10	15.20	7.35
7	on stove	10	10	10	10	16.80	7.68
8	next to plate	10	10	10	10	14.40	7.10
9	on wooden cabinet	10	10	8	10	17.80	7.26

从任务维度看，“next to ramekin”对 h8、h10 和 adaptive 都更敏感，说明物体相邻关系任务更容易受到路径细节、抓取姿态和放置误差影响。“wooden cabinet”在 h10 下下降到 8/10，但 adaptive 恢复到 10/10，说明动态缩短 horizon 对部分长路径或遮挡相关任务有帮助。另一方面，“on ramekin”adaptive 为 9/10，而固定 h6/h8/h10 均为 10/10，说明自适应阈值仍存在阶段误判可能，不能把 adaptive 结果夸大为全面优于固定 horizon。

7.11 探索类自适应 horizon 策略设计

固定实验显示，h6 稳定但推理次数较多，h10 高效但在少数任务中成功率下降。因此，探索题的目标不是让自适应策略在所有单项指标上超过所有固定 horizon，而是在不人工固定 horizon 的情况下，根据动作稳定性在线选择 h6、h8 或 h10。

理论上，chunk consistency 的定义是比较相邻两轮 action chunk 的重叠部分。若上一轮 chunk 为 A_{old} ，当前轮 chunk 为 A_{new} ，上一轮已执行 $offset$ 个动作，则可定义：

$$consistency_error = \text{mean} \left(\|A_{new}^i - A_{old}^{i+offset}\|_2 \right).$$

一致性误差越小，表示两轮对未来动作的预测越接近，可以选择更长 horizon；误差越大，则说明动作预测变化明显，应缩短 horizon，提高观测反馈频率。

本组在 `adaptive_horizon_policy.py` 中实现了严格 chunk-overlap consistency 接口, 同时考虑 gripper 状态变化。核心决策规则如下。

Listing 18: 自适应 horizon 三档阈值规则

```

1 if gripper_changed or consistency_error > tau_high:
2     horizon = min_horizon # h = 6
3 elif consistency_error > tau_low:
4     horizon = mid_horizon # h = 8
5 else:
6     horizon = max_horizon # h = 10

```

需要实事求是说明的是, 当前 StreamingVLA server 在真实 rollout 中逐步返回单个 action 增量, 并不直接向 client 暴露完整未来 action chunk。因此, 正式运行时使用 `executed_action_variation_proxy` 作为在线一致性代理: 计算最近已执行动作在前 6 个连续控制维度上的平均变化量, 作为动作片段是否稳定的信号。代码仍保留 chunk consistency 函数; 若后续 server 暴露完整 chunk, 可直接切换到严格重叠比较。

Listing 19: 已执行动作变化量代理信号

```

1 def executed_action_variation(actions, action_dims=6):
2     arr = np.asarray(list(actions), dtype=np.float32)
3     if arr.ndim != 2 or arr.shape[0] < 2:
4         return 0.0
5     dims = min(action_dims, arr.shape[1])
6     diffs = np.diff(arr[:, :dims], axis=0)
7     return float(np.linalg.norm(diffs, axis=1).mean())

```

7.12 自适应 horizon 实现过程

自适应实验在 `streamingvla.py` 中增加 `adaptive_horizon_mode` 入口。开启后, 脚本关闭 AEO judger, 初始化 `ChunkConsistencyAdaptiveHorizon`, 并在每轮 `replan` 前根据近期动作变化选择下一轮 horizon。每个 episode 记录 `selected_horizon_sequence`、`selected_horizon_mean/min/max`、`consistency_error_mean/max`、`gripper_change_count`、`adaptive_signal_source` 和 `result_source`。这些字段使探索实验不是概念描述, 而是可以被 CSV 逐条验证。

Listing 20: 自适应模式初始化与关闭 judger

```

1 if args.adaptive_horizon_mode:
2     args.use_judger = False
3     args.fixed_horizon_mode = True
4     adaptive_policy = ChunkConsistencyAdaptiveHorizon(
5         min_horizon=args.adaptive_min_horizon,
6         mid_horizon=args.adaptive_mid_horizon,
7         max_horizon=args.adaptive_max_horizon,
8         tau_low=args.adaptive_tau_low,
9         tau_high=args.adaptive_tau_high,
10    )

```

Listing 21: episode 级自适应结果写入 CSV

```

1 csv_writer.writerow({
2     "method": args.method_name,
3     "horizon": float(np.mean(selected_horizons)),
4     "inference_count": obs_times,
5     "task_completion_steps": t + 1,

```

```

6  "task_success": int(done),
7  "selected_horizon_sequence": ";".join(map(str, selected_horizons)),
8  "consistency_error_mean": float(np.mean(consistency_errors)),
9  "consistency_error_max": float(np.max(consistency_errors)),
10 "adaptive_signal_source": signal_source,
11 "result_source": "real_adaptive_rollout",
12 })

```

7.13 自适应 horizon 实验结果

自适应策略在 `libero_spatial` 上完成 100 episodes, CSV 中 `result_source` 全部为 `real_adaptive_rollout`, 说明结果来自真实平台评估而非离线模拟。总体结果如表8 所示。

表 8: 自适应 execution horizon 策略真实 rollout 结果

指标	数值
任务集	<code>libero_spatial</code>
episodes	100
成功 episode	98
总成功率	98%
平均 episode 时间	4.05 s
平均完成动作数	109.17
平均推理次数	15.77
平均端到端时延	10.95 ms
平均 selected horizon	7.10
consistency error mean	0.075
consistency error range	详见 CSV
在线信号来源	<code>executed_action_variation_proxy</code>

未成功 episode 共 2 个, 分别是 task 1 的“next to the ramekin”和 task 5 的“on the ramekin”。两者均运行到 220 步上限, 说明该现象不是日志中断, 而是完整 rollout 后任务未达成。task 1 的平均 selected horizon 为 7.66, mean error 为 0.060; task 5 的平均 selected horizon 为 6.59, mean error 为 0.097。后者误差较高且仍未达成任务成功判定, 说明动作变化代理信号虽然能触发保守 horizon, 但还不能完全识别所有精细接触阶段。

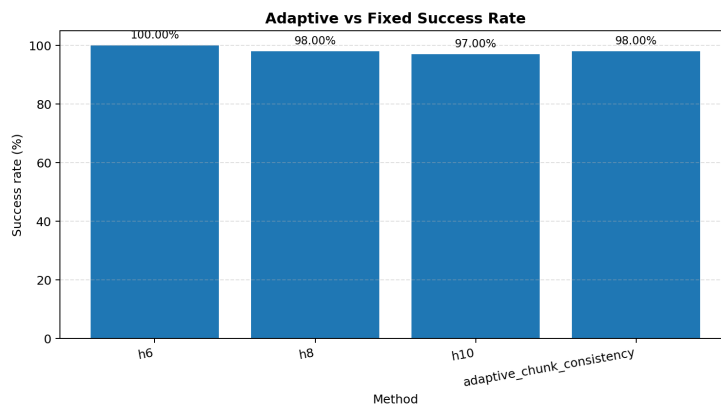


图 20: 自适应策略与固定 h6/h8/h10 成功率对比。adaptive 为 98%, 与 h8 持平, 高于 h10, 低于 h6。

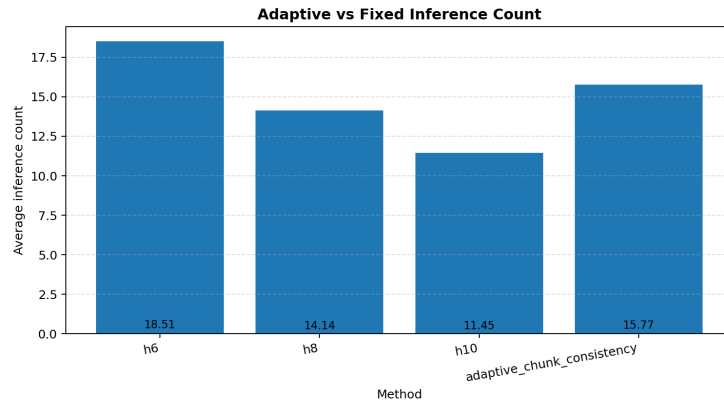


图 21: 自适应策略与固定 h6/h8/h10 平均推理次数对比。adaptive 为 15.77，少于 h6，多于 h8/h10。

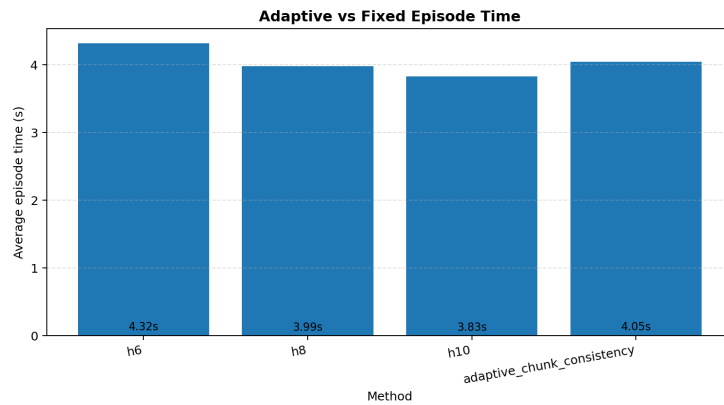


图 22: 自适应策略与固定 h6/h8/h10 平均 episode 时间对比。adaptive 位于 h6 与 h8/h10 之间。

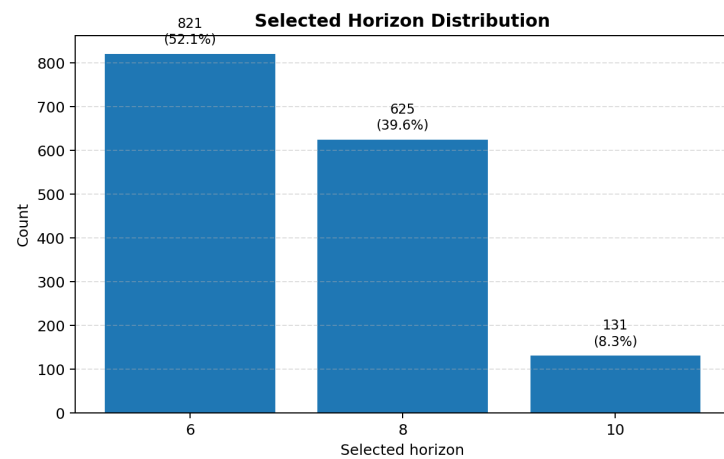


图 23: 自适应策略 selected horizon 分布。100 episodes 中共做出 1577 次选择，h6 占 52.1%，h8 占 39.6%，h10 占 8.3%。

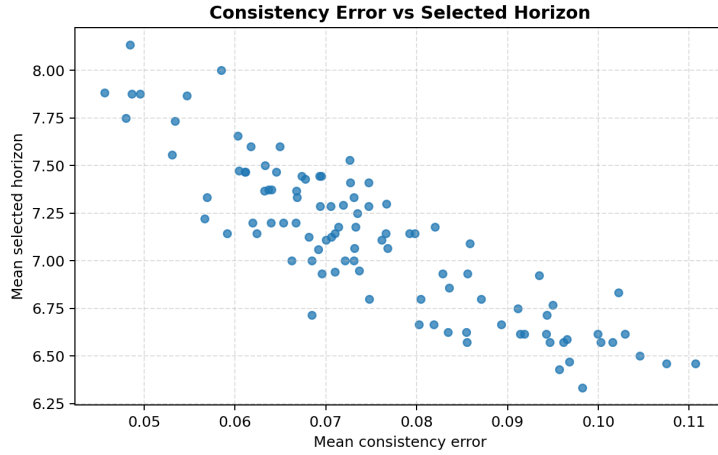


图 24: 每个 episode 的 consistency error mean 与 selected horizon mean 关系。误差较高时平均 horizon 倾向降低。

7.14 自适应结果横向与纵向分析

自适应 horizon 实验的目标不是在所有单项指标上超过固定 h6、h8 和 h10，而是在不人工指定固定执行长度的情况下，根据在线动作稳定性信号动态选择 execution horizon。实验结果显示，adaptive 策略在 100 episodes 中取得 98% 成功率，与 h8 持平，高于 h10，略低于最保守的 h6；平均推理次数为 15.77，少于 h6 的 18.51，说明该策略在保持较高成功率的同时减少了保守固定 horizon 带来的模型调用开销。

selected horizon 分布进一步说明策略没有退化为固定设置。100 episodes 中共做出 1577 次 horizon 选择，其中 h6 为 821 次，占 52.1%；h8 为 625 次，占 39.6%；h10 为 131 次，占 8.3%。这表明策略在多数阶段倾向于保持较高反馈频率，只在动作变化较平稳的片段选择更长 horizon。结合分任务结果看，adaptive 在“on wooden cabinet”任务中恢复到 10/10，而固定 h10 为 8/10，说明动态缩短 horizon 对部分空间关系和长路径任务具有实际帮助。

因此，本组自适应策略应定位为一个可运行的 horizon controller 原型。它没有改变模型权重、任务定义和成功率判定逻辑，而是在客户端执行层引入轻量级调度机制，实现了成功率与推理次数之间的在线折中。该结果说明，固定 horizon 的局限可以通过动态策略得到一定缓解，也为后续将完整 chunk-overlap consistency、gripper-aware 信号和 AEO 机制融合提供了实验基础。

8 StreamingVLA 相较 $\pi_{0.5}$ 的改进、优点与局限

8.1 执行流程差异

$\pi_{0.5}$ 的典型执行流程是：接收观察和语言指令，VLM 生成视觉语言表征，高层模块预测子任务，action expert 生成未来 H 步动作块，客户端执行动作块中的若干步，再重新观察。该流程能力强，但观察、生成、执行之间的串行等待导致停顿。

StreamingVLA 的流程变为：接收观察后，先生成可缓存的视觉语言前缀；action expert 不再等待完整动作块，而是按状态式 AFM 逐步生成动作增量并立即发送；当剩余动作对视觉变化的预测较小时，客户端提前开始下一次观察；若预测变化过大，则暂停提前观察，避免状态不一致。换言之， $\pi_{0.5}$ 解决“能否在开放环境中做对”，StreamingVLA 解决“在尽量不牺牲成功率的前提下能否更快、更连

续地做”。

8.2 性能收益

从论文数据看, StreamingVLA(AFM+AEO) 的平均成功率为 94.9%, 与 $\pi_{0.5}(h=10)$ 的 95.1% 接近; 单动作时间由 49.9 ms 降至 31.625 ms; 停顿由 230.8 ms 降至 36.0 ms。相较于 Naive EO, AEO 的成功率从 86.2% 回升到 94.9%, 说明自适应提前观察是保持性能的关键。

从本组复现实验看, $\pi_{0.5}$ 在 `libero_spatial` 上取得 100% 成功率, StreamingVLA baseline 取得 96% 成功率; 但 StreamingVLA 的平均 episode 时间由 7.48 s 降至 3.8 s, 平均动作速度由 13.92 actions/s 提升到 31.58 actions/s。这说明 StreamingVLA 在本组复现实验中也呈现出与论文一致的趋势: 以小幅成功率损失换取明显执行速度提升。

固定 horizon 实验进一步说明了系统级调度的重要性。h1、h2、h4 虽然反馈最频繁, 但成功率均为 0%, 且达到 220 步上限; h6、h8、h10 均能取得 97% 以上成功率, 并显著减少推理次数。由此可见, StreamingVLA 的收益不仅来自模型结构, 还来自观察、推理和动作执行之间的合理调度。

8.3 技术局限

StreamingVLA 的局限主要包括以下方面。

第一, 方法依赖 $\pi_{0.5}$ 等已有强 VLA 作为基础, 不直接解决开放世界语义泛化问题; 如果基础模型对任务理解错误, 流式化无法修复语义错误。第二, AEO predictor 需要额外训练和加载, 源码中 checkpoint 路径仍为占位, 复现门槛高于普通 openpi。第三, 提前观察阈值存在任务相关性, 阈值过低会减少重叠收益, 阈值过高会引入错误观察。第四, 状态式 AFM 要求数据中提供或预处理 `action_states`, 这改变了原始数据管道。第五, 本组固定 horizon 实验显示执行长度对成功率非常敏感, h1、h2、h4 甚至无法完成任务, 说明固定 horizon 不是一个通用最优策略。第六, 当前模型 `action_horizon=10` 使 h16 超出有效实验范围; 若要研究 h16, 需要修改模型动作块长度、服务端队列和相应 checkpoint。第七, 本组自适应 horizon 已完成真实 rollout, 但当前 server 未暴露完整未来 action chunk, 因此正式运行使用已执行动作变化量作为一致性代理信号; 该策略验证了在线调节的可行性, 但仍存在阈值敏感和阶段误判问题。

9 总结

本实验报告按照课程大作业要求完成了 π_0 、 $\pi_{0.5}$ 和 StreamingVLA 的论文源码研读、LIBERO 复现、固定 horizon profiling、自适应 horizon 探索、异常定位和结果可视化。 π_0 提供了 VLM 骨干与 flow matching 动作专家结合的基础, 使连续动作块生成成为可能; $\pi_{0.5}$ 在此基础上通过异构数据、高层子任务预测和后训练流程, 面向未见家庭环境扩展开放世界泛化能力; StreamingVLA 则针对 $\pi_{0.5}$ 的实时执行瓶颈, 把动作生成与执行、观察与执行重叠起来, 在 LIBERO 上以接近基线的成功率显著降低单动作时间和停顿间隔。

源码研读和复现过程表明, StreamingVLA 的创新不仅体现在论文中的 AFM 与 AEO 概念, 也落实到配置、数据、模型、策略封装和 websocket 服务端: `svla=True` 触发新模型类型, `action_states` 支撑状态式 AFM, `action_left_sum` 与 `threshold` 支撑 AEO, `streaming_infer` 和 streaming server 支撑动作逐步返回。复现中本组修复了 `policy.model` 属性访问、`use_sfp` 配置缺失、AEO predictor 路径、短 horizon 队列阻塞等问题, 并补充了固定 horizon 与 adaptive horizon 的 profiling 记录。上述修改没有改变模型主干、checkpoint 权重、任务定义和成功率判定逻辑, 主要服务于稳定复现、可

观测指标记录和 horizon 策略实验。

复现实验方面，本组完成了 $\pi_{0.5}$ 与 StreamingVLA baseline 各 100 episodes 的对比实验，结果为 $\pi_{0.5}$ 100% 成功、StreamingVLA baseline 96% 成功；StreamingVLA 的平均 episode 时间为 3.8 s，明显短于 $\pi_{0.5}$ 的 7.48 s。固定 horizon profiling 覆盖 baseline、h1、h2、h4、h6、h8、h10，其中 h6 成功率最高，h10 推理次数最少，h8/h10 在成功率和效率之间较均衡。结果显示，h1/h2/h4 并没有因为更频繁反馈而提升成功率，反而均未达到成功判定；h4 到 h6 之间出现明显跃迁，说明当前 StreamingVLA 的动作流式去噪需要足够 execution horizon 才能形成有效动作。

探索实验方面，本组实现了基于 chunk consistency 思路的自适应 horizon 策略，并完成 100 episodes。由于当前 server 只流式返回单步动作，正式运行采用 `executed_action_variation_proxy` 作为在线一致性代理信号，同时在代码中保留严格 chunk-overlap consistency 计算接口。adaptive 成功率为 98%，平均 selected horizon 为 7.10，h6/h8/h10 选择占比分别为 52.1%、39.6%、8.3%。与固定 h6/h8/h10 相比，它没有成为所有指标的绝对最优，但在保持较高成功率的同时减少了 h6 的推理次数，并避免了 h10 在部分任务上的成功率下降，体现了探索题要求的动态折中价值。

综合来看，本组实验不仅完成了课程要求的模型研读、源码分析和 LIBERO 平台复现，还围绕 StreamingVLA 的 execution horizon 给出了可复现、可解释的实验结论。固定 horizon profiling 揭示了当前 StreamingVLA 对执行长度具有明显结构性依赖：过短 horizon 会中断流式动作状态演化，h4 到 h6 之间存在有效下界；h6、h8、h10 则分别体现出稳定性、折中性和效率优势。自适应 horizon 实验进一步证明，在不修改模型权重的前提下，客户端可以利用在线动作稳定性信号进行动态执行长度调度，从而在较高成功率和较低推理次数之间取得折中。h16 边界验证则说明，execution horizon 的可选范围不仅由评估脚本参数决定，也受到模型 `action_horizon` 与服务端流式返回机制约束。上述结果共同构成了本次实验从复现、修复、profiling 到策略探索的完整闭环。

A 提交材料说明

本次提交压缩包除实验报告 PDF 外，还保留了支撑报告结论的图表、实验数据、运行日志和关键源码，便于教师或助教复核实验过程。为避免在正文中过多堆叠文件名，本报告仅对提交材料作概括说明；完整文件名和目录结构以压缩包中的 `README.txt` 与实际目录为准。

表 9: 提交材料概览

目录或文件	内容说明
实验报告.pdf、实验报告.tex	最终实验报告及其源文件，包含论文研读、源码分析、复现实验、固定 horizon profiling、自适应 horizon 探索和结果分析。
fig/	报告中使用的论文图表、固定 horizon 结果图、自适应 horizon 对比图和运行证据截图。
data/	$\pi_{0.5}$ 、StreamingVLA baseline、固定 horizon、自适应 horizon 的 CSV 结果、timing 文件、日志和统计汇总表。
source_code/	与报告分析直接相关的 openpi、StreamingVLA 关键源码，以及本组新增或修改的 profiling、fixed horizon、adaptive horizon 脚本。
README.txt	提交包目录说明、运行方式、结果文件位置和代码修改摘要。

上述材料能够对应报告中的主要结论：论文图表用于支撑模型结构和方法对比；CSV、timing 和日志用于支撑成功率、推理次数、完成步数和时延分析；源码文件用于说明本组对 StreamingVLA 的复现适配、固定 horizon profiling 和自适应 horizon 策略实现。

参考文献

- [1] Black, K., Brown, N., Darpinian, Z., et al. π_0 : *A Vision-Language-Action Flow Model for General Robot Control*. Provided PDF.
- [2] Black, K., Brown, N., et al. $\pi_{0.5}$: *a Vision-Language-Action Model with Open-World Generalization*. Provided PDF.
- [3] Shi, Y., Guo, D., Zhao, T., et al. *StreamingVLA: Streaming Vision-Language-Action Model with Action Flow Matching and Adaptive Early Observation*. Provided PDF.
- [4] Physical Intelligence. *openpi source repository*. Provided source archive `openpi-main(1).zip`.
- [5] Shi, Y. et al. *StreamingVLA source repository*. Provided source archive `StreamingVLA-main(1).zip`.
- [6] Liu, B., Zhu, Y., Gao, C., Feng, Y., Liu, Q., Zhu, Y., Stone, P. *LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning*. Cited by provided papers and source README.